23. ROS 2 TF2 Coordinate Transformation

> **pip Installation Note:** Ubuntu 24.04 marks the system Python as "externally managed" according to PEP 668, so installing directly with pip may report `externally-managed-environment`. `--break-system-packages` allows pip to install packages in this environment; a virtual environment usually does not need this argument, but this tutorial keeps it for consistent commands.

1. Introduction to TF2

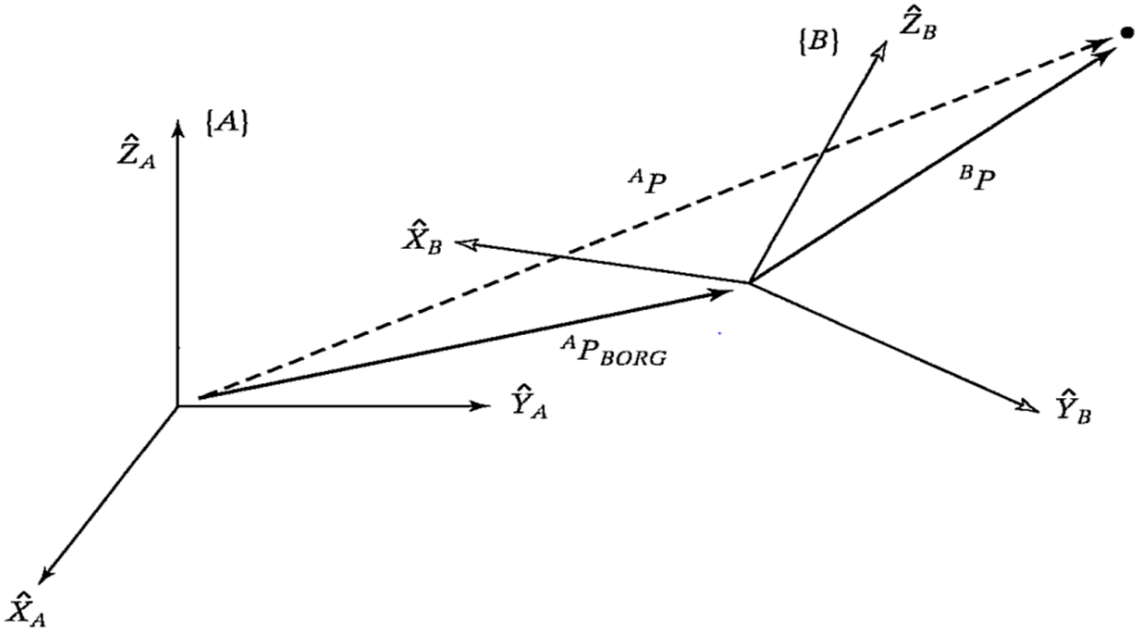
The coordinate frame is a concept we are very familiar with, and it is also an important foundation in robotics. In a complete robot system, there are many coordinate frames. How should the positional relationships between these coordinate frames be managed? ROS provides us with a powerful coordinate frame management tool: TF2.

TF system reference: [tf: The transform library | IEEE Conference Publication | IEEE Xplore]

2. Coordinate Frames in Robots

In a mobile robot system, coordinate frames are equally crucial. For example, the center point of a mobile robot is the base coordinate frame Base Link, the position of the LiDAR is called the laser coordinate frame laser link, and when the robot moves, the odometry accumulates the position. The reference frame of this position is called the odometry coordinate frame odom. The odometry also has accumulated errors and drift, and the reference frame of the absolute position is called the map coordinate frame map.

The relationships between coordinate frames layer by layer are complex. Some are relatively fixed, and some change constantly. The seemingly simple coordinate frames also become complex in the spatial range, so a good coordinate frame management system is particularly important.



The basic theory of coordinate frame transformation relationships is explained in every robotics textbook. It can be decomposed into two parts, **translation and rotation**, and is described by a four-by-four matrix. When coordinate frames are drawn in space, the transformation relationship between the two is actually a mathematical description of vectors.

The underlying principle of the TF function in ROS is the encapsulation of these mathematical transformations. For detailed theoretical knowledge, please refer to robotics textbooks. We mainly explain how to use the TF coordinate management system.

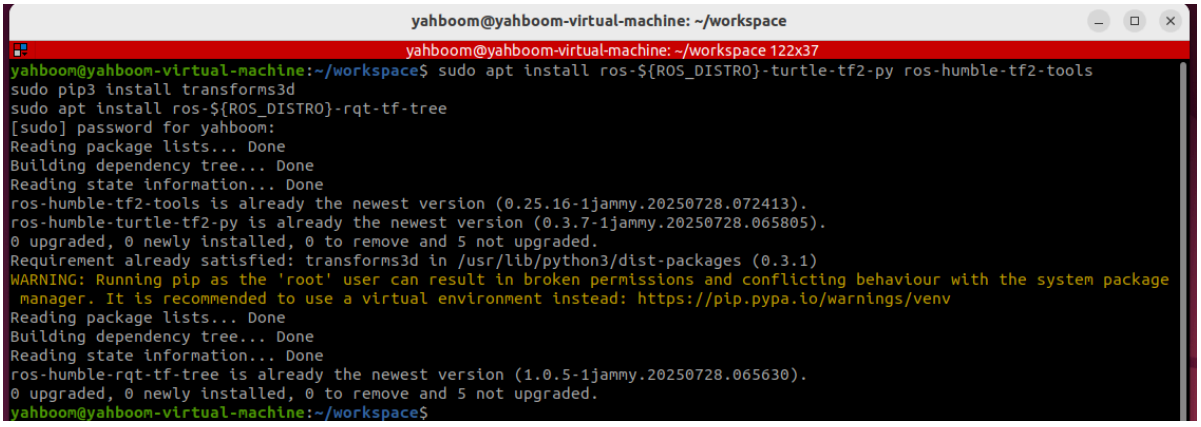
3. TF Command Line Operations

Let's first use an example with two turtles to understand a robot following algorithm based on coordinate frames. **For ease of demonstration, this lesson is best operated in a virtual machine**

3.1. Installing Related Tools

This example requires installing the corresponding package, the tf turtle simulator case, and the tf tree visualization tool.

```
sudo apt install ros-${ROS_DISTRO}-turtle-tf2-py ros-${ROS_DISTRO}-tf2-tools
sudo pip3 install transforms3d --break-system-packages
sudo apt install ros-${ROS_DISTRO}-rqt-tf-tree
```

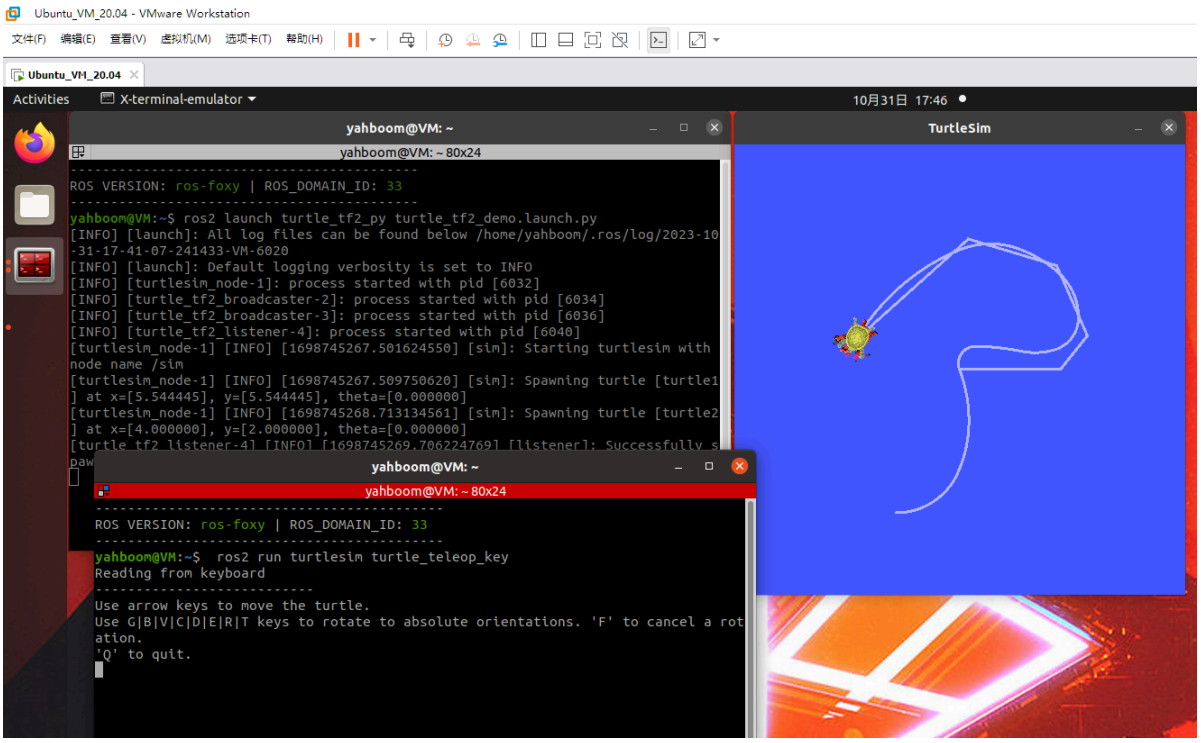


3.2. Starting Up

Then it can be started with a launch file. After that, we can control one of the turtles, and the other turtle follows automatically.

```
ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py
ros2 run turtlesim turtle_teleop_key
```

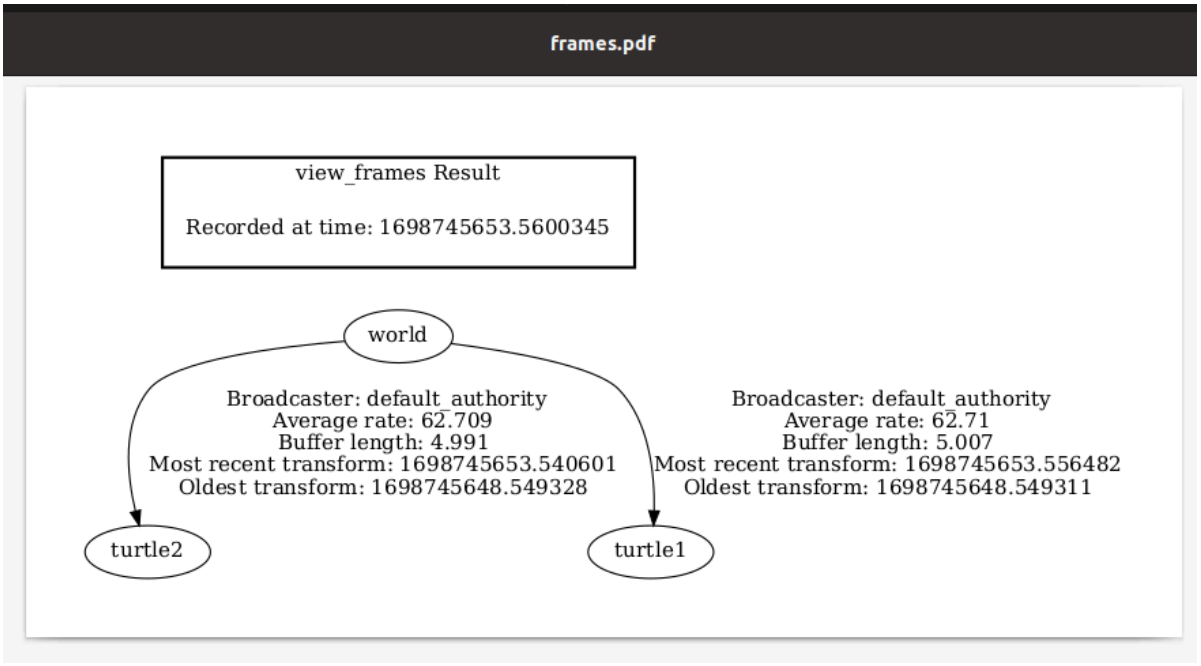
When we control one turtle to move, the other turtle also follows.



3.3. Viewing the TF Tree

```
ros2 run rqt_tf_tree rqt_tf_tree
```

The TF transformation tree can be seen in the rqt window.

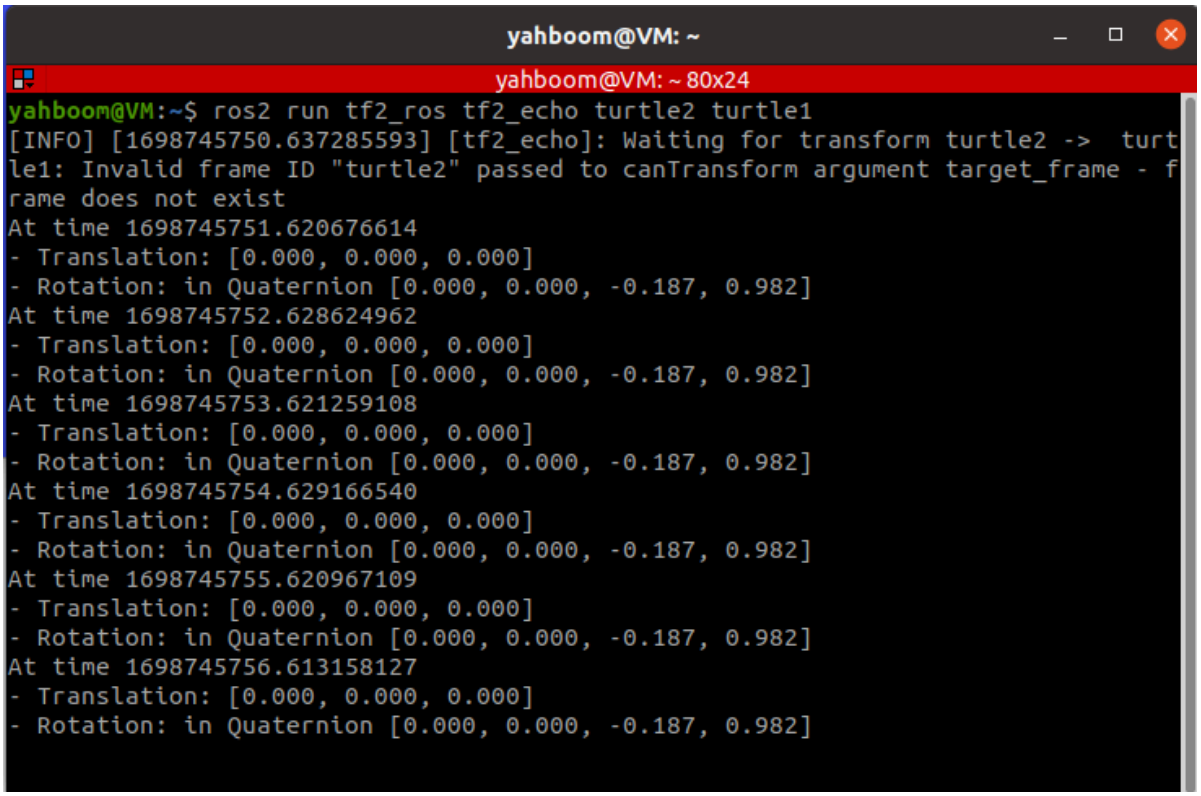


3.4. Querying Coordinate Transformation Information

Only seeing the structure of the coordinate frames is not enough. If we want to know the specific relationship between two coordinate frames, we can view it with the tf2_echo tool:

```
ros2 run tf2_ros tf2_echo turtle2 turtle1
```

After running successfully, the terminal prints the transformation values of the coordinate frames in a loop.

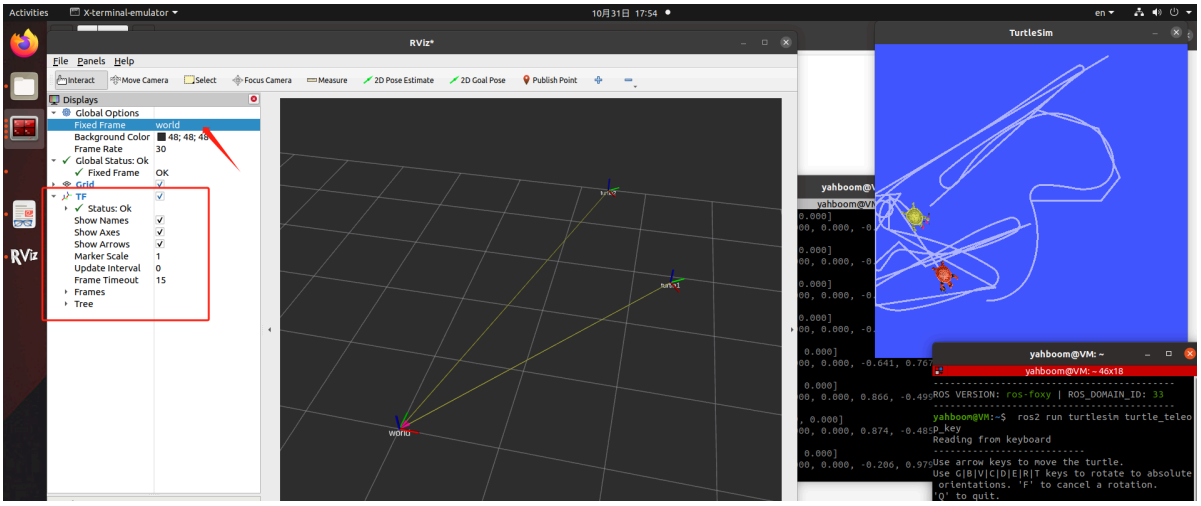


3.5. Coordinate Frame Visualization

- Run rviz2, and then add the TF display plugin.

```
rviz2
```

Set the reference coordinate frame in rivz2 to: world, add the TF display, and then make the turtle move. The coordinate axes in Rviz start to move. Isn't this more intuitive!



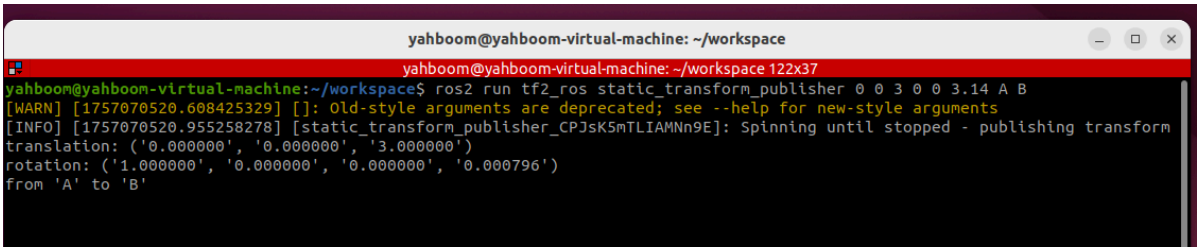
4. Static Coordinate Transformation

The so-called static coordinate transformation means that the relative position between two coordinate frames is fixed, such as the position between the LiDAR and base_link.

Example: **For ease of demonstration, this lesson is best operated in a virtual machine**

4.1. Publishing the Pose from A to B

```
ros2 run tf2_ros static_transform_publisher 0 0 3 0 0 3.14 A B
```



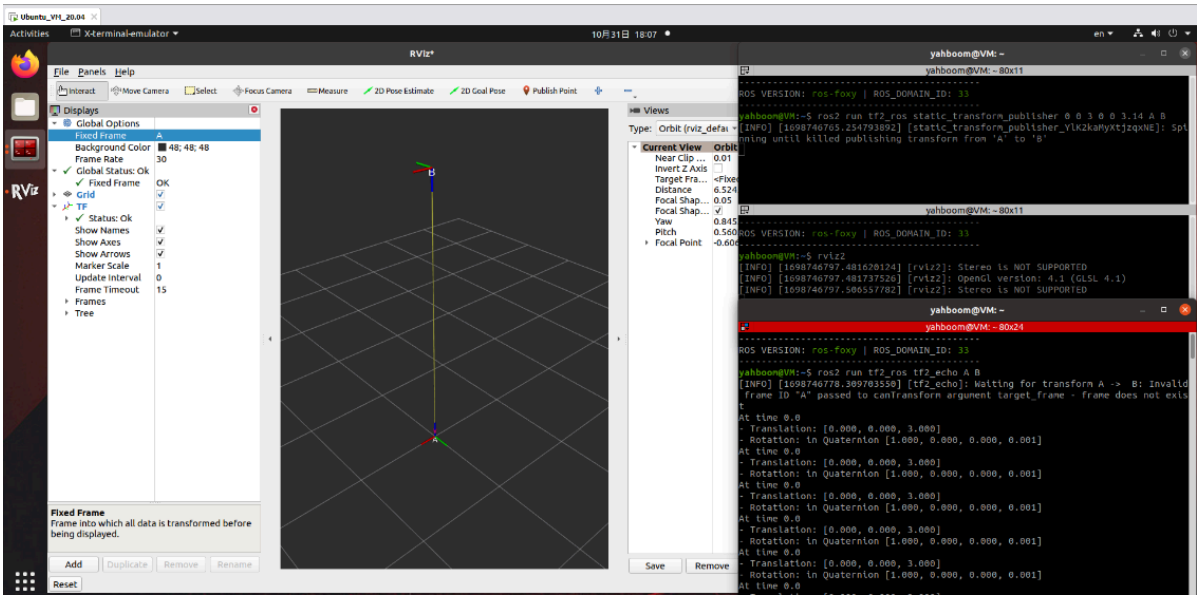
4.2. Listening/Obtaining the TF Relationship

```
ros2 run tf2_ros tf2_echo A B
```

4.3. RIVZ Visualization

- Run rviz2, and then add the TF display plugin.

```
rviz2
```



5. Case Introduction

In the previous lesson, the TF relationship in the turtle following case provided by the system was explained. In this lesson, we implement this function ourselves.

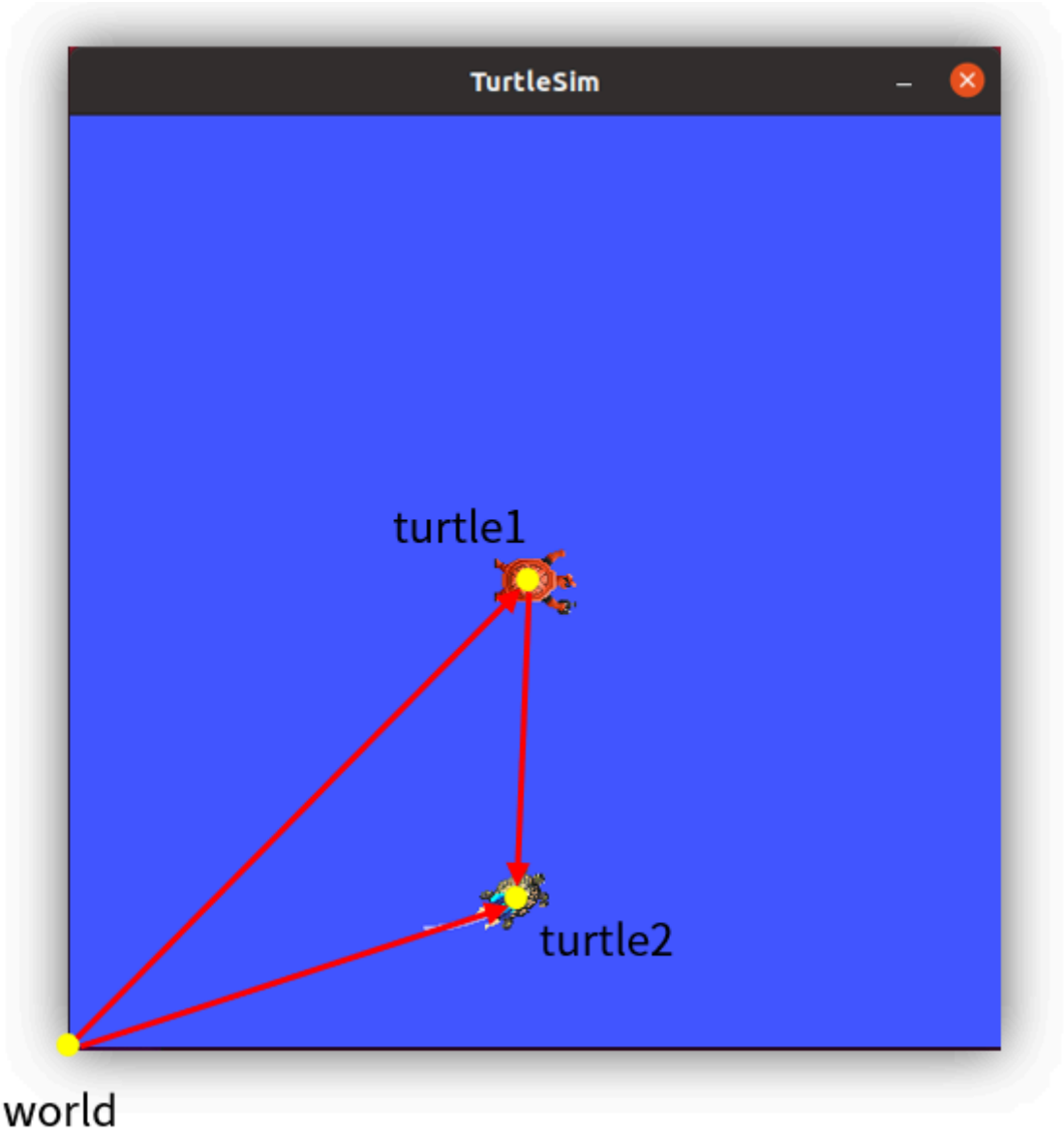
Course Content:

- Implementing the turtle following case through programming
- Mastering the implementation of a dynamic broadcaster through programming
- Mastering the implementation of listening to the coordinate transformation between coordinate frames through programming
- Mastering the use of PID control to convert physical quantities (distance, angle) into velocity control quantities

Advanced Content:

- Understanding the function of the TF system to transform coordinate frames across the time dimension

6. Principle Analysis of the Turtle Following Case



In the simulator with two turtles, we can define three coordinate frames. For example, the global reference frame of the simulator is called world, and the turtle1 and turtle2 coordinate frames are at the center points of the two turtles. In this way, the relative position of the turtle1 and world coordinate frames can represent the position of turtle 1, and the same applies to turtle 2.

To make turtle 2 move toward turtle 1, we draw a connecting line between them and add an arrow. How about that, does it remind you of the vector calculation learned in high school? We say that the description method of coordinate transformation is a vector, so in this following example, TF can solve it very well.

The length of the vector represents the distance, and the direction represents the angle. With the distance and the angle, we can calculate the velocity by setting an arbitrary time, can't we? Then comes the encapsulation and publication of the velocity topic, and turtle 2 can move.

So the core of this example is to implement the vector calculation through coordinate frames. The two turtles keep moving, so this vector also has to be calculated according to a certain period, which requires the dynamic broadcasting and listening of TF.

7. Creating a New Package

1. Create a new package in the src directory of the workspace to store our files.

```
ros2 pkg create pkg_tf --build-type ament_python --dependencies rclpy --node-name
turtle_tf_broadcaster
```

After executing the above command, the pkg_tf package is created, and a turtle_tf_broadcaster node is also created with the related configuration files already configured. Add the following code to the file 【turtle_tf_broadcaster.py】:

```
import rclpy                                        # ROS2 Python interface
library
from rclpy.node import Node                         # ROS2 node class
from geometry_msgs.msg import TransformStamped      # Coordinate transformation
message
import tf_transformations                           # TF coordinate transformation
library
from tf2_ros import TransformBroadcaster            # TF coordinate transformation
broadcaster
from turtlesim.msg import Pose                      # turtlesim turtle pose
message

class TurtleTFBroadcaster(Node):

    def __init__(self, name):
        super().__init__(name)                                  # Initialize the
parent class of the ROS2 node

        self.declare_parameter('turtlename', 'turtle')          # Create a
parameter for the turtle name
        self.turtlename = self.get_parameter(                   # Prefer the
parameter value set externally, otherwise use the default value
            'turtlename').get_parameter_value().string_value

        self.tf_broadcaster = TransformBroadcaster(self)        # Create and
initialize a broadcast object for TF coordinate transformation

        self.subscription = self.create_subscription(           # Create a
subscriber that subscribes to the pose message of the turtle
            Pose,
            f'/{self.turtlename}/pose',                         # Use the turtle
name obtained from the parameter
            self.turtle_pose_callback, 1)

    def turtle_pose_callback(self, msg):                                # Create a
callback function that processes the turtle pose message and converts the pose
message into a coordinate transformation
        transform = TransformStamped()                                  # Create a
message object of the coordinate transformation
```

```
        transform.header.stamp = self.get_clock().now().to_msg()      # Set the
timestamp of the coordinate transformation message
        transform.header.frame_id = 'world'                           # Set the
source coordinate frame of the coordinate transformation
        transform.child_frame_id = self.turtlename                    # Set the
target coordinate frame of the coordinate transformation
        transform.transform.translation.x = msg.x                     # Set the
translation along X, Y, and Z in the coordinate transformation
        transform.transform.translation.y = msg.y
        transform.transform.translation.z = 0.0
        q = tf_transformations.quaternion_from_euler(0, 0, msg.theta) # Convert
Euler angles to quaternions (roll, pitch, yaw)
        transform.transform.rotation.x = q[0]                         # Set the
rotation along X, Y, and Z in the coordinate transformation (quaternion)
        transform.transform.rotation.y = q[1]
        transform.transform.rotation.z = q[2]
        transform.transform.rotation.w = q[3]

        # Send the transformation
        self.tf_broadcaster.sendTransform(transform)     # Broadcast the
coordinate transformation; after the turtle pose changes, the coordinate
transformation information is updated in time

def main(args=None):
    rclpy.init(args=args)                                # Initialize the ROS2
Python interface
    node = TurtleTFBroadcaster("turtle_tf_broadcaster")  # Create and initialize
a ROS2 node object
    rclpy.spin(node)                                     # Wait in a loop for
ROS2 to exit
    node.destroy_node()                                  # Destroy the node
object
    rclpy.shutdown()                                     # Shut down the ROS2
Python interface
```

2. Next, create a file 【turtle_following.py】 in the same directory as turtle_tf_broadcaster.py, and add the following code:

```
import math
import rclpy                                          # ROS2 Python interface
library
from rclpy.node import Node                           # ROS2 node class
import tf_transformations                             # TF coordinate
transformation library
from tf2_ros import TransformException                # Exception class of TF
coordinate transformation
from tf2_ros.buffer import Buffer                     # Buffer class that
stores coordinate transformation information
from tf2_ros.transform_listener import TransformListener  # Listener class that
listens to coordinate transformations
from geometry_msgs.msg import Twist                   # ROS2 velocity control
message
from turtlesim.srv import Spawn                       # Service interface for
turtle spawning
```

```
class TurtleFollowing(Node):

    def __init__(self, name):
        super().__init__(name)                                  # Initialize
the parent class of the ROS2 node

        self.declare_parameter('source_frame', 'turtle1')       # Create a
parameter for the source coordinate frame name
        self.source_frame = self.get_parameter(                 # Prefer the
parameter value set externally, otherwise use the default value
            'source_frame').get_parameter_value().string_value

        self.tf_buffer = Buffer()                               # Create a
buffer that stores coordinate transformation information
        self.tf_listener = TransformListener(self.tf_buffer, self)  # Create a
listener for coordinate transformations

        self.spawner = self.create_client(Spawn, 'spawn')       # Create a
client that requests turtle spawning
        self.turtle_spawning_service_ready = False              # Flag of
whether the turtle spawning service has been requested
        self.turtle_spawned = False                             # Flag of
whether the turtle has been spawned successfully

        self.publisher = self.create_publisher(Twist, 'turtle2/cmd_vel', 1) #
Create the velocity topic of the following turtle

        self.timer = self.create_timer(1.0, self.on_timer)      # Create a
timer with a fixed period to control the motion of the following turtle

    def on_timer(self):
        from_frame_rel = self.source_frame                      # Source
coordinate frame
        to_frame_rel   = 'turtle2'                              # Target
coordinate frame

        if self.turtle_spawning_service_ready:                  # If the
turtle spawning service has been requested
            if self.turtle_spawned:                             # If the
following turtle has been spawned
                try:
                    now = rclpy.time.Time()                     # Get the
current time of the ROS system
                    trans = self.tf_buffer.lookup_transform(    # Listen to
the coordinate transformation from the source coordinate frame to the target
coordinate frame at the current moment
                        to_frame_rel,
                        from_frame_rel,
                        now)
                except TransformException as ex:                # If the
coordinate transformation fails to be obtained, enter the exception report
                    self.get_logger().info(
                        f'Could not transform {to_frame_rel} to {from_frame_rel}:
{ex}')
                    return
```

```
                msg = Twist()                                       # Create a
velocity control message
                scale_rotation_rate = 1.0                           # Calculate
the angular velocity according to the turtle angle
                msg.angular.z = scale_rotation_rate * math.atan2(
                    trans.transform.translation.y,
                    trans.transform.translation.x)

                scale_forward_speed = 0.5                           # Calculate
the linear velocity according to the turtle distance
                msg.linear.x = scale_forward_speed * math.sqrt(
                    trans.transform.translation.x ** 2 +
                    trans.transform.translation.y ** 2)

                self.publisher.publish(msg)                         # Publish the
velocity command, and the turtle follows
            else:                                                   # If the
following turtle has not been spawned
                if self.result.done():                              # Check
whether the turtle has been spawned
                    self.get_logger().info(
                        f'Successfully spawned {self.result.result().name}')
                    self.turtle_spawned = True
                else:                                               # The
following turtle is still not spawned
                    self.get_logger().info('Spawn is not finished')
        else:                                                       # If the
turtle spawning service has not been requested
            if self.spawner.service_is_ready():                     # If the
turtle spawning server is ready
                request = Spawn.Request()                           # Create
request data
                request.name = 'turtle2'                            # Set the
content of the request data, including the turtle name, xy position, and
orientation
                request.x = float(4)
                request.y = float(2)
                request.theta = float(0)

                self.result = self.spawner.call_async(request)      # Send the
service request
                self.turtle_spawning_service_ready = True           # Set the flag
to indicate that the request has been sent
            else:
                self.get_logger().info('Service is not ready')      # Prompt that
the turtle spawning server is not ready yet


def main(args=None):
    rclpy.init(args=args)                       # Initialize the ROS2 Python
interface
    node = TurtleFollowing("turtle_following")  # Create and initialize a ROS2
node object
    rclpy.spin(node)                            # Wait in a loop for ROS2 to exit
    node.destroy_node()                         # Destroy the node object
```

```
    rclpy.shutdown()                                # Shut down the ROS2 Python
interface
```

3. Create a launch folder in the pkg_tf package, create a file 【turtle_following.launch.py】 in the launch folder, and add the following content:

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():

    return LaunchDescription([
        DeclareLaunchArgument('source_frame', default_value='turtle1',
description='Target frame name.'),
        Node(
            package='turtlesim',
            executable='turtlesim_node',
        ),
        Node(
            package='pkg_tf',
            executable='turtle_tf_broadcaster',
            name='broadcaster1',
            parameters=[
                {'turtlename': 'turtle1'}
            ]
        ),
        Node(
            package='pkg_tf',
            executable='turtle_tf_broadcaster',
            name='broadcaster2',
            parameters=[
                {'turtlename': 'turtle2'}
            ]
        ),
        Node(
            package='pkg_tf',
            executable='turtle_following',
            name='listener',
            parameters=[
                {'source_frame': LaunchConfiguration('source_frame')}
            ]
        ),
    ])
```

8. Editing the Configuration File

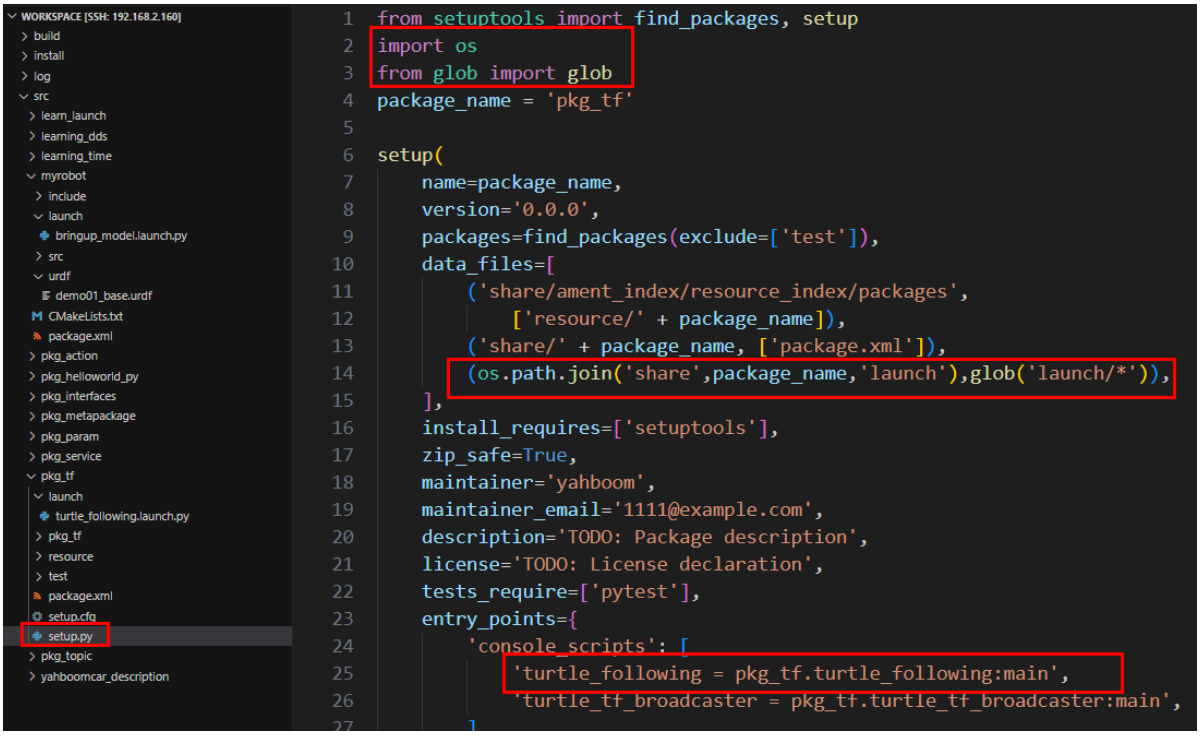
8.1. Configuration in setup.py

- Import the related libraries

```
import os
from glob import glob
```

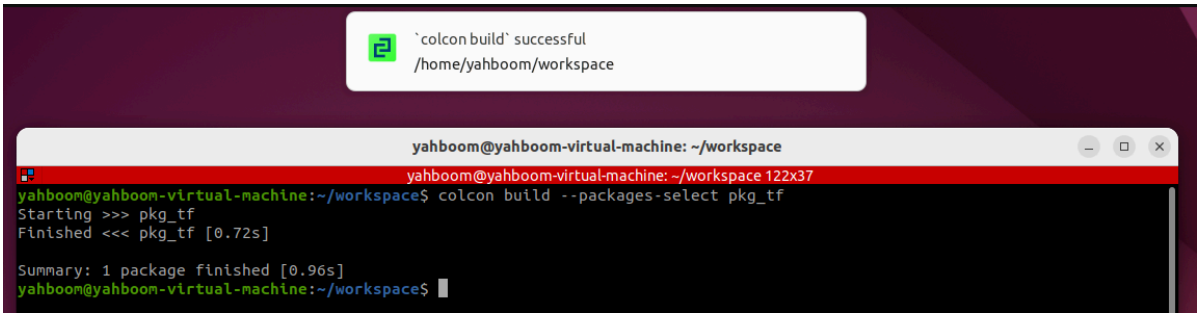
- Add the node information of turtle_following, and add the following command to copy the launch file to the shared directory in install

```
(os.path.join('share',package_name,'launch'),glob('launch/*')),
```



9. Building the Package

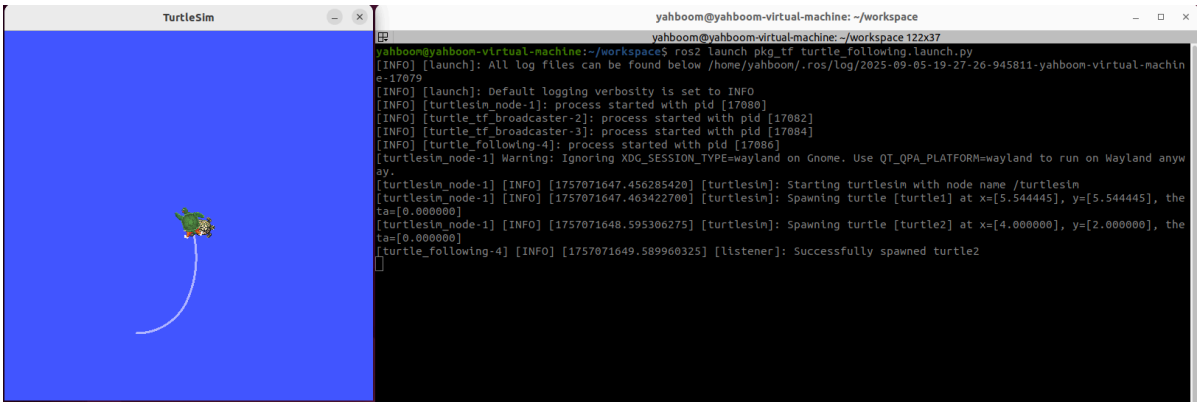
```
colcon build --packages-select pkg_tf
```



10. Running the Program

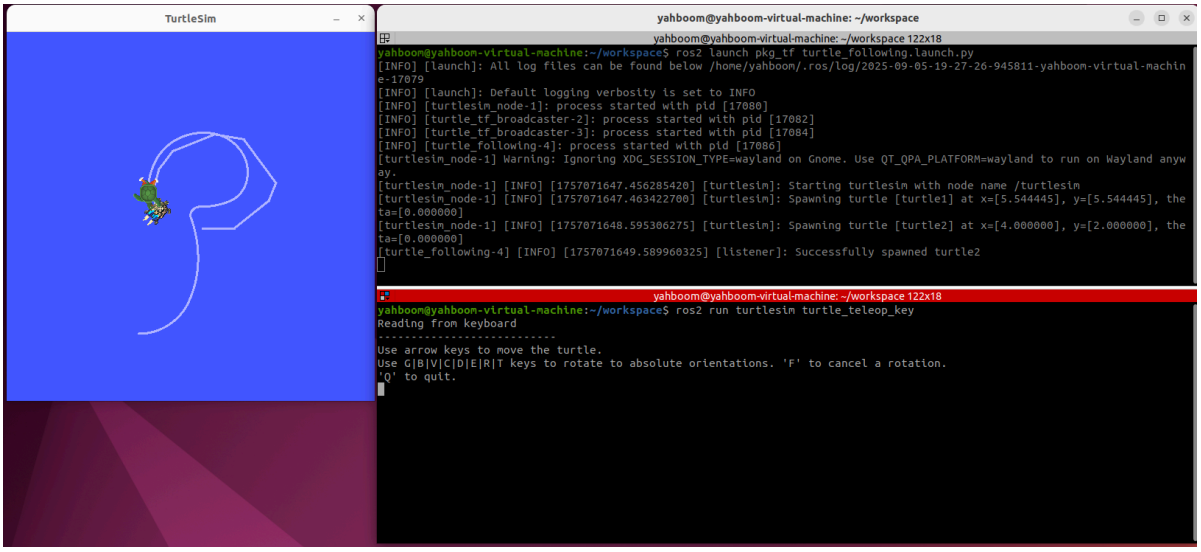
- Refresh the terminal environment variables, and then run

```
ros2 launch pkg_tf turtle_following.launch.py
```



- Start the turtle keyboard control node to control the first turtle, and the second turtle follows automatically

```
ros2 run turtlesim turtle_teleop_key
```



In this terminal, press the up, down, left, and right arrow keys on the keyboard to control one turtle to move, and then the other turtle follows until they overlap.

11. Advanced Content

- Understand the ability of TF to transform across the time dimension

  Buffer can automatically cache all TF transformation relationships in the TF system within the past 10 s through the buffer (any cache duration can be set by yourself through the Buffer constructor parameters). All transformations in the buffer are based on timestamps, and all transformations are continuously traceable. Even if the two coordinate frames are at different time points, the coordinate transformation relationship between them can still be found. For this point, you can refer to the reference document for designing the TF system, which explains the principle in detail (the document is in this lesson's folder, or through the link at the beginning of this section).

```
self.tf_buffer = Buffer()
self.tf_listener = TransformListener(self.tf_buffer, self)
```

- The following is a supplement to the turtle following case above. The red arrow indicates that the transformation between two coordinate frames at different time points can be found across time.

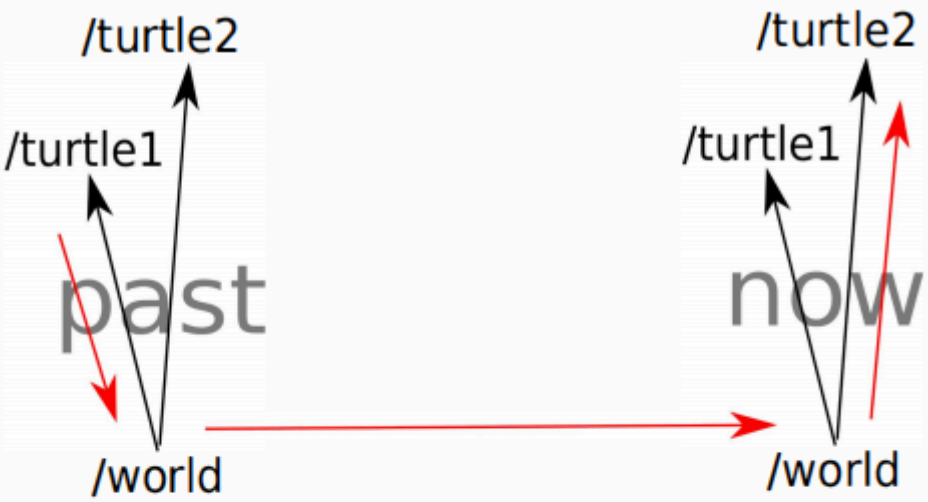


Fig. 4: A simple *tf* tree from a core ROS tutorial, with debugging information.

Long Term Data Storage: The above example assumes that you have the ability to transform between frames a and b at time 0 and b and c at time 1. This will only work if the length of the buffer in your *Listener* module is long enough to encompass both time 0 and time 1. To be able to store data for a long time, it should be transformed into the fixed frame, b, when saved. Then the source frame is the same as frame b and consequently the transform between the two is the identity, leaving only $T_{b@t_1}^{c@t_1}$ needing to be computed to find the current location of any previously observed object. This simple case is common in many robotic systems because they do not have the tools to compute the more complicated cases. This will only work when the assumptions about where the identity transform can be used is maintained.